Ben-Gurion University of the Negev

Faculty of Engineering Sciences

[DEPARTMENT DETAILS REQUIRED]

A DAG-Guided Framework for Proxy-State Effect Estimation in Irregular ICU Time Series

Thesis submitted in partial fulfillment of the requirements for the Master degree

[AUTHOR DETAILS REQUIRED]

[SUPERVISOR DETAILS REQUIRED]

[ADMINISTRATIVE DETAILS REQUIRED]

Approval Page

A DAG-Guided Framework for Proxy-State Effect Estimation in Irregular ICU Time Series

[AUTHOR DETAILS REQUIRED]

[SUPERVISOR DETAILS REQUIRED]

[DEPARTMENT DETAILS REQUIRED]

[APPROVAL TEXT REQUIRED]

[ADMINISTRATIVE DETAILS REQUIRED]

Secondary-Language Abstract

[STAGE 4 DRAFT REQUIRED]

[SUPERVISOR DECISION REQUIRED]

Primary-Language Abstract

[STAGE 4 DRAFT REQUIRED]

[SUPERVISOR DECISION REQUIRED]

Keywords

Primary-Language Keywords

[STAGE 4 DRAFT REQUIRED]

Secondary-Language Keywords

[STAGE 4 DRAFT REQUIRED]

[SUPERVISOR DECISION REQUIRED]

Acknowledgements

[STAGE 4 DRAFT REQUIRED]

[ADMINISTRATIVE DETAILS REQUIRED]

Contents

Abbreviations

[STAGE 4 DRAFT REQUIRED]

Notation and Symbols

[STAGE 4 DRAFT REQUIRED]

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation: Irregular ICU Time-Series Analysis

[STAGE 4 DRAFT REQUIRED]

1.2 Thesis Gap and Objective

[STAGE 4 DRAFT REQUIRED]

1.3 Research Questions and Contributions

[STAGE 4 DRAFT REQUIRED]

[SUPERVISOR DECISION REQUIRED]

1.4 Thesis Organization

[STAGE 4 DRAFT REQUIRED]

[SUPERVISOR DECISION REQUIRED]

Chapter 2

Background and Related Work

2.1 ICU EHR Datasets and Irregular Sampling

[STAGE 4 DRAFT REQUIRED]

2.2 Irregular Time-Series Representation Models

[STAGE 4 DRAFT REQUIRED]

2.3 Proxy Phenotyping and Weak Supervision

[STAGE 4 DRAFT REQUIRED]
[VALIDATION REQUIRED]

2.4 Causal Diagrams, Identification, HTE, Overlap, and Sensitivity

[STAGE 4 DRAFT REQUIRED]
[CITATION REQUIRED]

Chapter 3

Problem Definition and Study Design

3.1 Units, Time Horizons, and Data Objects

This thesis studies retrospective intensive-care records represented as irregular clinical time series. The operational study unit is a time-series record or ICU stay, depending on the source dataset and preprocessing path. PhysioNet 2012 source files use record identifiers, whereas MIMIC-III source tables use ICU-stay and admission identifiers. The implemented pipelines normalize these source-specific identifiers to the canonical join key `ts_id`. This identifier should be read as the analysis-record identifier used by the software; it is not assumed to be a globally unique person identifier across repeated admissions or stays.

For a study unit i, the observed longitudinal data are represented as a finite collection of measurement events

$$\mathcal{O}_i = \{(i, t_{ij}, v_{ij}, x_{ij}) : j = 1, \ldots, n_i\},$$

where i is the normalized `ts_id`, t_{ij} is elapsed time in minutes from record start or ICU admission, $v_{ij} \in \mathcal{V}$ is the observed variable name, and x_{ij} is the numeric value recorded for that variable. In the causal repository this long-format object is stored in `ts` with the required columns `ts_id`, `minute`, `variable`, and `value`. Static or baseline quantities such as age, sex or gender coding, height, weight, and ICU-type indicators are represented within the same event schema as variables observed at or near the beginning of the record when the preprocessing implementation provides them.

The event representation is intentionally irregular. Observations occur at nonuniform times, different variables are measured with different frequencies, and most variables are absent at most possible times. A missing measurement is therefore not interpreted as a normal value. It may reflect a clinical decision not to measure, an unrecorded observation, a source-table limitation, or a preprocessing exclusion. The thesis treats informative measurement and missingness processes as important threats to interpretation, not as causal findings established by the repository.

The primary patient- or stay-level outcome used by the downstream causal repository is `in_hospital_mortality`. In the causal processed artifact, this outcome is stored in `oc`, the outcome table keyed by `ts_id`. The third object in that processed artifact, `ts_ids`, is the sorted collection of normalized identifiers represented in `ts`. Later predictive processing uses a

distinct split-aware artifact and must not be conflated with this causal contract.

3.2 Prediction Task

The prediction task is a multi-label proxy-state prediction problem over irregular ICU time series. For each study unit i and proxy state k, let $L_{ik}^{rule} \in \{0, 1\}$ denote a rule-derived proxy label. These labels are clinically inspired analytical constructs generated from observed data and deterministic source-code rules. They are weak clinical labels or proxy phenotypes, not chart-adjudicated diagnoses, manually verified clinical states, or validated phenotypes unless separate validation evidence is supplied in a later chapter.

The supervised predictive workflow estimates a vector of proxy-state labels from the observed time-series representation. Its target matrix is loaded from a latent-label CSV keyed by `ts_id`; the supervised proxy-label targets are not taken from the mortality column in `oc`. For model m, the exported prediction object may contain probabilities $\hat{p}_{ik}^{(m)}$ and corresponding binary predicted labels $\hat{L}_{ik}^{(m)}$. These outputs are model estimates of rule-derived proxy labels rather than direct measurements of patient physiology.

At the level of study design, five analytical tasks are distinguished. First, rule-based proxy-state construction derives binary proxy labels from observed clinical measurements. Second, multi-label proxy-state prediction estimates those rule-derived labels from irregular time-series inputs. Third, proxy-label aggregation may combine binary outputs from several voter sources into a single proxy-state table. Fourth, mortality prediction or association analysis evaluates whether proxy-state representations contain prognostic information about `in_hospital_mortality`. Fifth, observational causal-effect estimation treats selected proxy-state indicators as modeled exposures and estimates adjusted contrasts under explicit assumptions. Detailed proxy rules, model architectures, estimator algorithms, and numerical results are intentionally deferred to later chapters.

3.3 Causal Question, Exposures, Outcome, Estimands, Assumptions

The downstream causal-analysis setting is retrospective and observational. No intervention is assigned by this study. The repository uses software parameters named `treatment` for selected binary `LAT_*` columns, but many such columns represent illness-state or organ-dysfunction proxy states rather than well-defined administered therapies. In the thesis narrative these variables are therefore described as proxy-state exposures or modeled treatment variables unless a later section explicitly defines a manipulable intervention.

Let $A_i \in \{0, 1\}$ denote a selected proxy-state exposure, Y_i denote the binary outcome `in_hospital_mortality`, W_i denote observed adjustment variables selected from a project-authored DAG and available columns, and X_i denote effect-modifier features used to describe heterogeneity. Potential-outcome notation such as $Y_i(a)$ is useful as conceptual language, but

causal interpretation requires more than the presence of a graph or an estimator. It depends on assumptions about the causal graph, consistency, exchangeability conditional on observed covariates, positivity or overlap, outcome and exposure measurement, and unmeasured confounding [1, 2, 3].

Accordingly, later estimates should be interpreted as adjusted effect estimates under project assumptions, not as unconditional clinical effects. The existence of a DAG, matching script, or CATE estimator does not prove identification. Proxy states may be useful for structured analysis while still carrying label noise, temporal ambiguity, and intervention-definition limitations.

[SUPERVISOR DECISION REQUIRED: final estimand wording for proxy-state exposures and aggregated CATE summaries]

The approved main research question is how irregular ICU time-series data can be converted into clinically interpretable proxy-state representations and used, under explicit causal assumptions, to support DAG-guided adjusted effect estimation for in-hospital mortality. Within the scope of this chapter and the next, the relevant subquestions are whether rule-derived clinical proxy states can be predicted from irregular ICU time series, which data contracts are needed to connect PhysioNet 2012, MIMIC-III, STraTS, and the causal-analysis pipeline, whether proxy-state representations contain mortality-related information, how adjusted estimates vary across proxy-state exposures or patient characteristics, and how robust those estimates are to modeling and confounding assumptions. These are study questions and design commitments; their answers are reserved for the results and discussion chapters.

Chapter 4

Data and Preprocessing

4.1 PhysioNet 2012 Pipeline

The PhysioNet/Computing in Cardiology Challenge 2012 dataset provides ICU time-series records and associated in-hospital mortality outcomes for a mortality-prediction challenge setting [4]. In this thesis it functions as one of the two ICU time-series sources used to exercise the proxy-state prediction and downstream causal-analysis pipeline. The local repository does not track the raw challenge files or a verified processed-data pickle, so this section describes the implemented preprocessing contract rather than asserting final cohort counts.

[RESULT REQUIRED: final number of included PhysioNet records after preprocessing]

The causal-repository PhysioNet preprocessing implementation traverses the raw `set-a`, `set-b`, and `set-c` folders and reads one patient-record file at a time. It removes rows with missing parameter names, skips records with at most five retained measurement rows, and removes observations with negative values, which the source code treats as missingness indicators. It converts source times from `HH:MM` strings into elapsed minutes, renames `RecordID` to `ts_id`, `Parameter` to `variable`, and `Value` to `value`, and reads outcome files by mapping `In-hospital_death` to `in_hospital_mortality`. After concatenating the source sets, it drops duplicate events and converts the categorical `ICUType` measurement into one-hot-style variables `ICUType_1` through `ICUType_4`.

The resulting causal processed artifact is serialized as three objects: `ts`, `oc`, and `ts_ids`. The `ts` object is a long event table with `ts_id`, `minute`, `variable`, and `value`. The `oc` object contains `ts_id`, `length_of_stay`, `in_hospital_mortality`, and a source subset indicator. The `ts_ids` object is derived from identifiers observed in `ts`. This artifact is the expected input to rule-based tagging, mortality-from-proxy analysis, matching, and CATE estimation in the causal repository.

4.2 MIMIC-III Pipeline

MIMIC-III is a critical-care electronic health-record database used here as the second ICU data source [5]. The implemented workflow follows a clinical time-series benchmark style in that it

transforms raw ICU events into stay-level time-series examples suitable for prediction, while adapting the output contract for this thesis pipeline [6]. The repository does not establish a tracked raw-data snapshot, extraction date, or final processed artifact hash.

[RESULT REQUIRED: final number of included MIMIC-III ICU stays after preprocessing]

The active causal-repository MIMIC preprocessing implementation reads broad categories of MIMIC-III source tables: ICU-stay timing from `ICUSTAYS`, demographics from `PATIENTS`, bedside measurements from `CHARTEVENTS`, laboratory measurements from `LABEVENTS`, fluid outputs from `OUTPUTEVENTS`, administered inputs from `INPUTEVENTS_CV` and `INPUTEVENTS_MV`, and mortality/timing information from `ADMISSIONS`. Large event tables are processed in chunks and written to temporary shards before feature rows are collected. The implementation filters to adult ICU stays, removes chart events marked with an error flag, requires valid chart times and numeric or textual values, applies item-ID mappings and range filters for selected variables, converts selected units, and assigns events without direct ICU-stay identifiers to an ICU interval when possible.

For the causal contract, event times are converted to elapsed minutes relative to ICU admission, events are restricted to the early ICU window used by the implementation, age and gender are added as static rows at minute zero, and duplicate (`ts_id`, `minute`, `variable`) events are collapsed by averaging. The helper contract canonicalizes stay identifiers, normalizing numeric-looking identifiers such as `12345.0` to `12345`. It defines the canonical `ts` columns as `ts_id`, `minute`, `variable`, and `value`, defines the canonical `oc` columns as `ts_id`, `length_of_stay`, `in_hospital_mortality`, and `subset`, and intentionally excludes internal MIMIC identifiers such as `HADM_ID` and `SUBJECT_ID` from the exported outcome table.

Inspection found a source-level issue that should be resolved before relying on local MIMIC preprocessing execution: the active script reads `ADMISSIONS.csv` with `DEATHTIME` for early-death filtering, while the canonical outcome helper requires `HOSPITAL_EXPIRE_FLAG` to construct `in_hospital_mortality`. This does not change the documented target contract, but it is a reproducibility risk recorded in the Stage 4.1 evidence report and deferred-fix register.

4.3 STraTS Split-Aware Artifacts Versus Causal Artifacts

The predictive STraTS repository uses a different processed-data contract from the causal repository. Its loader expects a five-object pickle consisting of `events`, `oc`, `train_ids`, `val_ids`, and `test_ids`, where the final three objects explicitly encode train, validation, and test identifiers. The loader accepts stay identifiers named `ts_id`, `icustay_id`, or `ICUSTAY_ID`, canonicalizes them to `ts_id`, and validates the split arrays after the same normalization. This split-aware contract is therefore not interchangeable with the causal repository's unsplit `[ts, oc, ts_ids]` artifact.

In supervised STraTS runs, the latent-label CSV is joined to the selected split identifiers after ID normalization. The loader filters labels to the supervised IDs, raises an error when

labels are missing for any supervised `ts_id`, raises an error for duplicate normalized labels, and uses all non-`ts_id` columns in the latent CSV as proxy-label targets. The mortality column in `oc` is not used as the supervised proxy-label target. Prediction export writes `ts_id`, one probability column per proxy state, and one thresholded binary column per proxy state; the wrapper scripts inspected for this stage export the `all` split for their prediction CSVs, although final archive-copy provenance remains incomplete.

The predictive loader also implements split-aware leakage controls. It keeps only variables observed in the training split, derives normalization statistics from the training split, computes static-feature normalization from training rows, and validates label alignment before training. For MIMIC-III supervised prediction it restricts events to the first 24 hours. The pretraining path removes test data, uses training-derived variable statistics, uses a 48-hour maximum window for PhysioNet 2012, and permits MIMIC-III observations up to five days while sampling 24-hour input windows. These controls reduce some sources of leakage, but they do not guarantee absence of leakage, nor do they resolve missing raw-data and split-provenance limitations.

The parent router can bridge contracts by reading a causal `[ts, oc, ts_ids]` artifact and constructing a STraTS-style `[ts, oc, train_ids, val_ids, test_ids]` payload through a seeded identifier shuffle. That bridge is an orchestration convenience and does not prove how every archived final STraTS artifact was generated.

Table 4.1: Processed artifact contracts used by the causal and STraTS repositories.

Property	Causal repository	STraTS repository
Processed artifact	`[ts, oc, ts_ids]`	`[events, oc, train_ids, val_ids, test_ids]`
Split representation	Unsplit ID collection	Explicit train/validation/test IDs
Main use	Tagging and causal analysis	Predictive training and export
Identifier convention	Canonical `ts_id`	Loader-normalized `ts_id`
Proxy-label source	Separate latent CSV downstream	Separate latent CSV joined to split IDs

[FIGURE REQUIRED: audited dataflow diagram linking raw data, causal artifacts, STraTS artifacts, proxy labels, and causal analysis]

Several provenance limitations remain. Raw PhysioNet and MIMIC-III data are not tracked in Git. The processed pickles used by final runs are external or absent from the audited repository. Some run records contain machine-specific absolute paths, and final cohort counts require verified processed artifacts or manifests. Clean-checkout reproduction therefore requires authorized data access, artifact hashes, and a documented split-generation record.

[VALIDATION REQUIRED: checksums for processed PhysioNet and MIMIC-III dataset artifacts]

[VALIDATION REQUIRED: provenance of train/validation/test split generation for final STraTS artifacts]

Chapter 5

Clinical Proxy-State Construction

5.1 Rationale and Terminology

[STAGE 4 DRAFT REQUIRED]

[VALIDATION REQUIRED]

5.2 PhysioNet Proxy-State Rules

[STAGE 4 DRAFT REQUIRED]

[TABLE REQUIRED]

5.3 MIMIC Proxy-State Rules and Validation Artifacts

[STAGE 4 DRAFT REQUIRED]

[VALIDATION REQUIRED]

[TABLE REQUIRED]

5.4 Predicted and Majority-Vote Proxy States

[STAGE 4 DRAFT REQUIRED]

[VALIDATION REQUIRED]

Chapter 6

Predictive Modeling of Proxy States

This chapter describes the predictive layer that estimates rule-derived proxy-state vectors from ICU time-series records. The supervised target for this layer is a proxy-label matrix loaded from a separate CSV file keyed by `ts_id`; the inputs are irregular measurement histories plus static covariates prepared under the data contracts described in Chapter 4. The models therefore predict analytical proxy labels rather than adjudicated clinical states.

The implementation compares STraTS-family irregular-observation models with recurrent, convolutional, attention-based, decay-based, and interpolation-based baselines. This chapter defines the representation, training, evaluation, and export mechanics needed to connect these predictors to the later proxy-aggregation workflow. Final run configurations, selected result artifacts, and numerical model comparisons are intentionally deferred to Chapters 9 and 10.

6.1 Self-Supervised STraTS Pretraining

STraTS represents an irregular time series as a set of observed measurement tokens rather than as a fully imputed grid. The implemented STraTS class constructs each token by adding three learned components: a continuous embedding of measurement time, a continuous embedding of the normalized measurement value, and an embedding of the variable identity. The token sequence is contextualized with a transformer-style block and then pooled with fusion attention to obtain a fixed-dimensional time-series representation [7]. Static covariates are combined with this representation before either the self-supervised forecasting head or the supervised proxy-label head is applied.

The same source file implements both `strats` and `istrats` model types. In the verified implementation, `strats` pools contextualized observation embeddings and passes static covariates through a learned demographic embedding. The `istrats` branch instead pools the initial triplet embeddings and concatenates the raw static-covariate vector. The pretraining command-line guard permits self-supervised pretraining only for `strats` and `istrats`; the recurrent, convolutional, attention-grid, decay, and interpolation baselines do not have a supported self-supervised pretraining path in the current source.

The pretraining dataset loader reads the split-aware processed artifact and removes held-out

test identifiers before constructing unsupervised examples. It builds the variable vocabulary from variables observed in the pretraining train split, derives value means and standard deviations from that train split, and saves the resulting variable list, normalization table, and maximum time horizon to `pt_saved_variables.pkl`. For PhysioNet 2012 the maximum time is 48 hours. For MIMIC-III the loader permits observations up to five days but samples at most a 24-hour input context and clamps implausibly old age values in the same way as the supervised loader.

Each self-supervised example is formed by sampling a forecast anchor from observed timestamps that are not the final timestamp and are at least 12 hours after record start. Observations up to the anchor form the historical context, truncated by the maximum observation count and, for MIMIC-III, by the 24-hour context limit. The target window extends two hours after the anchor. For each variable observed in that future window, the implementation keeps the last observed normalized value and sets the corresponding forecast mask entry to one; variables without future observations are masked out of the loss. Input times are rescaled to the interval $[-1, 1]$ using the dataset-specific maximum time.

The pretraining head maps the combined time-series and static representation to one forecast value per variable. Its objective is masked squared error over variables observed in the sampled future window, so unobserved future variables do not contribute to the loss. The pretraining evaluator materializes batches for each evaluation split, sampling three forecast batches per evaluation chunk when a split is first evaluated, and reports `loss_neg`, the negative masked loss, for checkpoint selection. The best pretraining checkpoint is written as `checkpoint_best.bin`; loss values themselves are not reported in this methods chapter.

Checkpoint-loaded supervised training is implemented by constructing the target supervised architecture, loading the supplied pretraining checkpoint, and copying overlapping tensors into the current model state before calling `load_state_dict`. When `--load_ckpt_path` is supplied, the supervised dataset also loads `pt_saved_variables.pkl` from the same directory to reuse the pretraining variable vocabulary, normalization statistics, and maximum-time metadata. This source path supports warm-started STraTS-family training, but the archived final exports still lack a complete manifest linking each exported CSV to the intended pretraining checkpoint, supervised checkpoint, split artifact, and archive-copy step.

[VALIDATION REQUIRED: recover a predictive manifest linking each final STraTS-family supervised checkpoint, pretraining checkpoint, export command, and exported CSV]

6.2 Supervised Multi-Label Models

The supervised prediction task uses a latent-label CSV as its target matrix. The loader accepts identifier columns named `ts_id`, `icustay_id`, or `ICUSTAY_ID`, canonicalizes them to `ts_id`, and checks consistency when more than one accepted identifier column is present. It then filters the label table to the supervised split identifiers, raises an error for missing labels, raises an error for duplicate normalized identifiers, sorts rows to match the internal supervised identifier

order, and treats every non-`ts_id` column as a proxy-label target. The number and order of supervised targets are therefore determined at runtime by the CSV schema.

All supervised backends share a multi-label output contract. During training the model returns binary cross-entropy with logits against the full target vector, using per-target positive-class weights computed from the supervised train split. During evaluation or prediction export, the same model call without labels returns sigmoid probabilities. Static covariates are normalized from training rows and are concatenated, either after a learned demographic embedding or directly in the `istrats` branch, with each model's time-series representation. In scratch supervised training the binary head maps the combined representation directly to the proxy-label logits. In checkpoint-loaded training the source inserts an intermediate forecasting head before the binary head so that overlapping pretraining tensors can be reused.

The STraTS supervised backend uses the irregular observation triplets described in Section 6.1: time, value, and variable embeddings, transformer contextualization, fusion attention, and static-feature combination [7]. The `istrats` branch shares the same triplet construction and fusion-attention module, but its verified pooling and static-feature path differ as described above. The current final artifact archive contains STraTS prediction CSVs, while an iSTraTS final prediction artifact was not found during this stage.

The GRU baseline uses a dense hourly grid rather than the raw irregular token set. For each hour and variable, the loader builds value, observation-mask, and elapsed-time-since-observation channels, mean-fills unobserved values from training data, normalizes the filled values, and concatenates the three channel groups before passing them to a gated recurrent unit [8]. The final recurrent hidden state is concatenated with the static-feature embedding and projected through the shared supervised head.

The GRU-D backend uses a model-specific irregular sequence representation containing observed values, missingness masks, elapsed-time tensors, sequence lengths, and static covariates [9]. The implemented cell updates last observed values, applies learned exponential decay to inputs and hidden states as a function of elapsed time, and includes missingness masks in the recurrent gates. The representation passed to the supervised head is the final valid recurrent state for each sequence combined with the static-feature embedding.

The TCN baseline uses the same dense hourly value, observation-mask, and elapsed-time representation as the GRU baseline, but processes the sequence with temporal convolutional residual blocks [10]. The implementation permutes the dense grid into channel-first form, applies a temporal convolutional network, takes the last temporal embedding, and concatenates it with the static-feature embedding before projection to proxy-label logits.

The SAnD backend also consumes the dense hourly representation. It applies a one-dimensional input embedding, learned positional encoding, masked attention blocks constrained by a local attention window, and dense interpolation over a fixed number of reference positions before combining the resulting time-series representation with static covariates [11]. In this repository, SAnD is therefore an attention-based dense-grid comparator rather than an irregular-token model.

InterpNet is implemented as an interpolation-prediction baseline for irregularly sampled time series [12]. The loader supplies observation times in hours, value matrices, observation masks, and holdout masks. The model performs single-channel interpolation, cross-channel interpolation, concatenates interpolation-derived quantities, and feeds the resulting sequence to a GRU before applying the shared supervised head. When labels are present, the implementation adds an auxiliary reconstruction loss over held-out observed values. Implementation presence does not establish an approved final InterpNet performance artifact; no final InterpNet prediction CSV or training summary was found in the inspected final archive.

[RESULT REQUIRED: approved final InterpNet evaluation artifact before inclusion in the predictive performance comparison]

Table 6.1: Implemented predictive model families and non-numeric evidence status.

Model	Temporal representation and irregularity handling	Static features and pretraining path	Citation and repository evidence status
STraTS	Irregular observation tokens built from time, value, and variable embeddings; observation masks handle padding, and fusion attention pools contextualized tokens.	Learned demographic embedding is concatenated with the pooled representation; self-supervised pretraining is supported.	[7]. Implementation confirmed; wrappers and archived STraTS outputs present; final checkpoint-to-export manifest incomplete.
iSTraTS	Same initial triplet construction as STraTS, but fusion attention pools initial triplet embeddings rather than contextualized embeddings.	Raw static-covariate vector is concatenated; self-supervised pretraining is supported.	[7]. Implementation confirmed; no approved final iSTraTS export found in inspected archive.
GRU	Dense hourly grid with value, observation-mask, and elapsed-time channels; unobserved values are mean-filled before normalization.	Learned demographic embedding is concatenated with the final recurrent state; pretraining is not supported by the current guard.	[8]. Implementation confirmed; archived GRU prediction CSVs present; export provenance incomplete.
GRU-D	Irregular sequence of value, mask, elapsed-time, and sequence-length tensors; learned decays and masks enter the recurrent update.	Learned demographic embedding is concatenated with the final valid recurrent state; pretraining is not supported by the current guard.	[9]. Implementation confirmed; archived GRU-D prediction CSVs present; export provenance incomplete.

Model	Temporal representation and irregularity handling	Static features and pretraining path	Citation and repository evidence status
TCN	Dense hourly grid with value, observation-mask, and elapsed-time channels processed by temporal convolutional residual blocks.	Learned demographic embedding is concatenated with the last convolutional embedding; pretraining is not supported by the current guard.	[10]. Implementation confirmed; archived TCN prediction CSVs present; export provenance incomplete.
SAnD	Dense hourly grid with input embedding, positional encoding, masked attention, and dense interpolation; mask/delta channels come from the loader.	Learned demographic embedding is concatenated after dense interpolation; pretraining is not supported by the current guard.	[11]. Implementation confirmed; archived SAnD prediction CSVs present; export provenance incomplete.
InterpNet	Irregular times, values, observation masks, and hold-out masks are transformed by single-channel and cross-channel interpolation before a GRU.	Learned demographic embedding is concatenated with the GRU representation; pretraining is not supported by the current guard.	[12]. Implementation confirmed; no approved final result artifact found in inspected archive.

The supervised evaluator computes metrics independently for each target column in the requested split. Target columns with only one observed class in that split are skipped. For the remaining targets, it computes AUROC, area under the precision-recall curve, and the maximum value of the pointwise minimum of precision and recall. The reported split-level values are simple averages over the non-degenerate target columns, with loss and negative loss also recorded. During supervised training, checkpoint selection uses the validation value of `auprc + auroc`; this validation score should not be interpreted as a test score.

Training mechanics are shared across the supervised backends at the top-level script. The script constructs a deterministic seed by adding the run index to the configured seed, uses AdamW for trainable parameters, clips gradient norms at 0.3, evaluates at the configured validation schedule, and saves `checkpoint_best.bin` when the validation selection metric improves. If an `eval_train` split is evaluated, it is the first 2000 training examples in the current supervised split object, which makes it a diagnostic subset rather than a separate external evaluation set.

6.3 Prediction Export and Normalization

Prediction export is requested with `--save_pred_csv_path`. The export routine can run after a training run or in an export-only invocation with no training steps scheduled. Immediately before export, the script reloads `checkpoint_best.bin` from the current output directory only if that file exists. The exported split is selected by `--predict_split`, whose accepted values are `train`, `val`, `test`, and `all`; in the `all` case the exporter iterates over all supervised identifiers in the train, validation, and test split object. The inspected wrapper scripts and archived export logs support `predict_split=all` for the final prediction CSVs, but they do not provide a complete processed-artifact, checkpoint, and archive-copy manifest.

For every exported row, the exporter removes labels from the batch, obtains probability outputs from the model, thresholds each probability at 0.5, and writes a combined CSV. The first column is `ts_id`. It is followed by one probability column named `<label>_prob` for each target and then one thresholded binary prediction column named exactly as the target label. These binary columns are model-predicted proxy labels, not rule-derived labels and not clinical adjudications.

Table 6.2: Prediction export schema before downstream voter normalization.

Column group	Naming pattern	Meaning	Source of value
Identifier	`ts_id`	Normalized supervised record identifier.	STraTS dataset loader and split object.
Probability outputs	`<label>_prob`	Sigmoid model probability for a proxy-label target.	Supervised model called without labels.
Binary predictions	`<label>`	Thresholded model-predicted proxy label.	Probability output thresholded at 0.5.

The combined prediction CSV is not the same file shape required by downstream voter aggregation. The helper `split_predicted_latent_tags.py` validates that `ts_id` is the first column, expects an even number of non-identifier columns, splits the first half into probability columns and the second half into binary tag columns, and writes separate probability and absolute-tag CSVs. The majority-vote input contract is stricter: each voter file must contain `ts_id` plus binary proxy columns, with all voter files sharing the same latent-column set after validation. The parent router can collect STraTS outputs, drop probability columns, enforce the approved latent ordering, and write normalized voter CSVs for later aggregation. The semantics of the majority-vote proxy label belong to Chapter 5; here the important point is the predictive handoff contract.

[VALIDATION REQUIRED: record the final export split, source processed-pickle hash, split-generation seed or ID manifest, source checkpoint, and archive-copy provenance for each prediction CSV]

Chapter 7

Causal Graph and Effect-Estimation Methodology

7.1 Dataset-Specific DAGs

[STAGE 4 DRAFT REQUIRED]

[SUPERVISOR DECISION REQUIRED]

7.2 Adjustment-Set Logic

[STAGE 4 DRAFT REQUIRED]

[VALIDATION REQUIRED]

7.3 Matching and Heterogeneous-Effect Estimators

[STAGE 4 DRAFT REQUIRED]

[CITATION REQUIRED]

[SUPERVISOR DECISION REQUIRED]

Chapter 8

Robustness, Sensitivity, and Validation Design

8.1 Overlap and Support Diagnostics

[STAGE 4 DRAFT REQUIRED]

[RESULT REQUIRED]

[VALIDATION REQUIRED]

8.2 Sensitivity and Robustness Values

[STAGE 4 DRAFT REQUIRED]

[VALIDATION REQUIRED]

8.3 Permutation Checks and Reproducibility Validation

[STAGE 4 DRAFT REQUIRED]

[VALIDATION REQUIRED]

Chapter 9

Experimental Design

9.1 Prediction Experiment Matrix

[STAGE 4 DRAFT REQUIRED]

[RESULT REQUIRED]

[VALIDATION REQUIRED]

9.2 Causal Experiment Matrix

[STAGE 4 DRAFT REQUIRED]

[SUPERVISOR DECISION REQUIRED]

[VALIDATION REQUIRED]

Chapter 10

Results

10.1 Data and Cohort Summary

[RESULT REQUIRED]

[VALIDATION REQUIRED]

10.2 Proxy Prevalence and Co-Occurrence

[RESULT REQUIRED]

[VALIDATION REQUIRED]

10.3 Predictive Performance

[RESULT REQUIRED]

[VALIDATION REQUIRED]

10.4 Learning Curves

[RESULT REQUIRED]

[VALIDATION REQUIRED]

10.5 Mortality Prediction From Proxy States

[RESULT REQUIRED]

[VALIDATION REQUIRED]

10.6 Matching Results

[RESULT REQUIRED]

[VALIDATION REQUIRED]

10.7 CATE Estimates

[RESULT REQUIRED]

[SUPERVISOR DECISION REQUIRED]

[VALIDATION REQUIRED]

10.8 Heterogeneity Diagnostics

[RESULT REQUIRED]

[VALIDATION REQUIRED]

10.9 Overlap and Support

[RESULT REQUIRED]

[VALIDATION REQUIRED]

10.10 Sensitivity

[RESULT REQUIRED]

[VALIDATION REQUIRED]

10.11 Permutation

[RESULT REQUIRED]

[VALIDATION REQUIRED]

10.12 Cross-Dataset Comparison

[RESULT REQUIRED]

[SUPERVISOR DECISION REQUIRED]

[VALIDATION REQUIRED]

Chapter 11

Discussion

11.1 Answer Research Questions

[STAGE 4 DRAFT REQUIRED]

[RESULT REQUIRED]

[VALIDATION REQUIRED]

11.2 Limitations and Threats to Validity

[STAGE 4 DRAFT REQUIRED]

[RESULT REQUIRED]

[VALIDATION REQUIRED]

Chapter 12

Conclusions and Future Work

12.1 Contribution Summary and Future Work

[STAGE 4 DRAFT REQUIRED]

[RESULT REQUIRED]

Appendix A

Complete Proxy-State Definitions

[STAGE 4 DRAFT REQUIRED]

[VALIDATION REQUIRED]

Appendix B

DAG Node and Edge Inventory

[STAGE 4 DRAFT REQUIRED]

[SUPERVISOR DECISION REQUIRED]

Appendix C

Configuration Fields and Resolved Run Settings

[STAGE 4 DRAFT REQUIRED]

[VALIDATION REQUIRED]

Appendix D

Model Hyperparameters and Training Commands

[STAGE 4 DRAFT REQUIRED]

[VALIDATION REQUIRED]

Appendix E

Additional Figures and Tables

[STAGE 4 DRAFT REQUIRED]

[FIGURE REQUIRED]

[TABLE REQUIRED]

[VALIDATION REQUIRED]

Appendix F

Reproducibility and Environment Notes

[STAGE 4 DRAFT REQUIRED]

[VALIDATION REQUIRED]

Appendix G

Legacy Code and Excluded Artifacts

[STAGE 4 DRAFT REQUIRED]

[VALIDATION REQUIRED]

Bibliography

[1] Judea Pearl. "Causal Diagrams for Empirical Research." In: *Biometrika* 82.4 (1995), pp. 669–688. DOI: 10.1093/biomet/82.4.669. URL: https://doi.org/10.1093/biomet/82.4.669.

[2] Miguel A. Hernan and James M. Robins. "Using Big Data to Emulate a Target Trial When a Randomized Trial Is Not Available." In: *American Journal of Epidemiology* 183.8 (2016), pp. 758–764. DOI: 10.1093/aje/kwv254. URL: https://doi.org/10.1093/aje/kwv254.

[3] Miguel A. Hernan and Sarah L. Taubman. "Does Obesity Shorten Life? The Importance of Well-Defined Interventions to Answer Causal Questions." In: *International Journal of Obesity* 32.Suppl 3 (2008). Author/academic repost used because publisher PDF was not directly downloadable in this environment., S8–S14. DOI: 10.1038/ijo.2008.82. URL: https://doi.org/10.1038/ijo.2008.82.

[4] Ikaro Silva et al. "Predicting In-Hospital Mortality of ICU Patients: The PhysioNet/Computing in Cardiology Challenge 2012." In: *Computing in Cardiology*. Vol. 39. 2012, pp. 245–248. URL: https://physionet.org/content/challenge-2012/1.0.0/.

[5] Alistair E. W. Johnson et al. "MIMIC-III, a freely accessible critical care database." In: *Scientific Data* 3.160035 (2016). DOI: 10.1038/sdata.2016.35. URL: https://doi.org/10.1038/sdata.2016.35.

[6] Hrayr Harutyunyan et al. "Multitask Learning and Benchmarking with Clinical Time Series Data." In: *Scientific Data* 6.1 (2019), p. 96. DOI: 10.1038/s41597-019-0103-9. URL: https://doi.org/10.1038/s41597-019-0103-9.

[7] Sindhu Tipirneni and Chandan K. Reddy. "Self-Supervised Transformer for Sparse and Irregularly Sampled Multivariate Clinical Time-Series." In: *ACM Transactions on Knowledge Discovery from Data* (2022). DOI: 10.1145/3516367. URL: https://doi.org/10.1145/3516367.

[8] Kyunghyun Cho et al. "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation." In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*. 2014, pp. 1724–1734. DOI: 10.48550/arXiv.1406.1078. arXiv: 1406.1078. URL: https://arxiv.org/abs/1406.1078.

[9] Zhengping Che et al. "Recurrent Neural Networks for Multivariate Time Series with Missing Values." In: *Scientific Reports* 8.6085 (2018). DOI: 10.1038/s41598-018-24271-9. URL: https://doi.org/10.1038/s41598-018-24271-9.

[10] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling." In: *arXiv* (2018). DOI: 10.48550/arXiv.1803.01271. arXiv: 1803.01271 [cs.LG]. URL: https://arxiv.org/abs/1803.01271.

[11] Huan Song et al. "Attend and Diagnose: Clinical Time Series Analysis using Attention Models." In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2018. URL: https://ojs.aaai.org/index.php/AAAI/article/view/11843.

[12] Satya Narayan Shukla and Benjamin M. Marlin. "Interpolation-Prediction Networks for Irregularly Sampled Time Series." In: *International Conference on Learning Representations*. 2019. URL: https://openreview.net/forum?id=r1efr3C9Ym.